

Simple VESS

Victor Shoup
Offchain Labs
victor@shoup.net

June 20, 2025

Abstract

We present a scheme for verifiably encrypting a Shamir secret sharing to a committee of shareholders. Such a scheme can be used to easily implement distributed key generation (DKG) and resharing protocols used in threshold signing and decryption protocols. Our scheme is a minor variation on known techniques, and is not the most efficient in terms of communication and computational complexity. However, it is extremely simple and easy to implement. Moreover, for moderately sized shareholder committees of up to, say, 13 parties or so, and for applications where a DKG/resharing only needs to be performed occasionally, its performance should be acceptable in practice.

1 Introduction

Roughly speaking, a scheme for **verifiably encrypting a secret sharing**, or **VESS** scheme, allows one party to encrypt shares of a Shamir secret sharing to a committee of shareholders, so that any party (not necessarily a shareholder) can publicly verify that it has correctly done so. A VESS scheme is a useful building block in designing **distributed key generation (DKG)** and **resharing** protocols that can then be used as a part of a **threshold signing or decryption protocol**. It is especially well suited for this application, as it allows for DKG/resharing protocols that are very simple, require very little interaction, and require no synchrony assumptions.

In this brief note we present a very simple VESS protocol. It is a minor variation on known techniques, and is not the most efficient in terms of communication and computational complexity. However, it is extremely simple and easy to implement, making use of nothing more than operations in standard cryptographic groups used for discrete-logarithm-based schemes, and standard cryptographic hash functions. Moreover, for moderately sized shareholder committees of up to, say, 13 parties or so, and for applications where a DKG/resharing only needs to be performed occasionally, its performance should be acceptable in practice.

2 Preliminaries

We assume n parties $P_1, \dots, P_n$, of which up to f may be corrupt. We assume static corruptions. Let $\mathcal{C}$ denote the set of corrupt parties and $\mathcal{H}$ the set of honest parties. We

want to perform secret sharing with a reconstruction threshold of t . In typical applications, we have $t = f + 1$; however, other applications may require a higher threshold, such as $n - f$.

We gave a group $\mathbb{G}$ of prime order q generated by $g \in \mathbb{G}$, written using multiplicative notation.

For $\mathbf{C} = (C_0, \dots, C_{t-1}) \in \mathbb{G}^t$ and $\beta \in \mathbb{Z}_q$, we define

$$\mathbf{C}^{(\beta)} := C_0 C_1^\beta \cdots C_{t-1}^{\beta^{t-1}}.$$

Given two such vectors $\mathbf{C}, \mathbf{D} \in \mathbb{G}^t$, we define their product $\mathbf{C} \cdot \mathbf{D} \in \mathbb{G}^t$ to be their *componentwise product*. We can similarly define their quotient, $\mathbf{C}/\mathbf{D} \in \mathbb{G}^t$ — again, this is defined componentwise.

For a polynomial $\omega = \alpha_0 + \alpha_1 x + \cdots + \alpha_{t-1} x^{t-1} \in \mathbb{Z}_q[x]_{<t}$, and for a group element $u \in \mathbb{G}$, we define

$$u^\omega := (u^{\alpha_0}, u^{\alpha_1}, \dots, u^{\alpha_{t-1}}) \in \mathbb{G}^t.$$

With these conventions, we have

$$(u^\omega)^{(\beta)} = u^{\omega(\beta)}.$$

A VESS scheme works as follows. We assume that all parties have public keys $pk_1, \dots, pk_n$. Each honest party P_j will generate a public/secret-key pair independently using a key-generation algorithm. Corrupt parties can choose their keys arbitrarily — even depending on the public keys of the honest parties. Note that unlike in other works on VESS, such as [Gro21, KMM⁺23], we do *not* require users to prove possession of their secret keys. This distinction may be important in settings where we want to carry out a simulation based security analysis (for example, in the UC framework) — the most practical schemes for proof of possession require “rewinding” the adversary, which may be problematic in such settings.

An **encryption algorithm** takes as input a polynomial $\omega = \alpha_0 + \alpha_1 x + \cdots + \alpha_{t-1} x^{t-1} \in \mathbb{Z}_q[x]_{<t}$ and associated data ad , and outputs a **dealing**, which is of the form $(\mathbf{C}, \chi)$, where $\mathbf{C} := g^\omega \in \mathbb{G}^t$, and χ represents a collection of encryptions of $\omega(j)$ under pk_j for $j \in [n]$. A **dealing verification algorithm** takes as input $(\mathbf{C}, \chi, ad)$ and outputs **accept** or **reject**. A **decryption algorithm** takes as input $(\mathbf{C}, \chi, ad)$ along with the index $j \in [n]$ of a recipient P_j and their secret key sk_j , and outputs $\mu \in \mathbb{Z}_q$ such that $\mathbf{C}^{(j)} = g^\mu$ or **fail**.

We require both a **soundness** property and a **privacy** property.

Soundness: If $(\mathbf{C}, \chi, ad)$ passes verification, then the probability that decryption fails for any honest recipient is negligible.

Privacy: The privacy property we want is AD-only CCA security. Roughly speaking, this means that if an honest party encrypts ω under associated data ad , yielding $(\mathbf{C}, \chi)$, then provided the adversary never subsequently submits a decryption request on behalf of an honest P_j under the same ad , then the adversary learns nothing more than $\mathbf{C}$ and $\{\omega(j)\}_{j \in \mathcal{C}}$.

Of course, such a scheme should also satisfy a **correctness** property, which says that an honestly generated dealing should always pass verification and decrypt correctly.

In the intended application of DKG and resharing, each party P_i will generate a dealing, and these will be combined using an ACS protocol (that is, a protocol for agreement on a common set). In this application, P_i will generate a dealing using a randomly chosen polynomial ω with associated data ad that includes

- a *party ID* uniquely associated with party P_i ,
- a *protocol instance ID* uniquely associated with the DKG or resharing protocol instance to which this dealing belongs.

The first component of ad is essential for security, in order to isolate dealings from different parties from one another. The second component is less essential, but serves to isolate protocol instances from one another. This is good security practice and is also essential to obtain a level of “proactive” security, in which the same party may be viewed as corrupt in one protocol instance, but honest in another.

We also note that to obtain proactive security, it is essential to rotate keys with some regular frequency. This certainly includes the encryption keys associated of the VESS. It should likely also include any signing keys associated with consensus and any other protocols.

Using the soundness, privacy, and correctness properties of the VESS scheme, one can prove the above is a secure DKG protocol, but with one caveat: the adversary may bias the resulting public key to a limited extent. For some applications, such bias is completely benign, while for others, it is not. See Appendix A for more details.

3 A multi-recipient encryption scheme

An essential building block in our VESS scheme is a **multi-recipient encryption (MRE)** scheme, which allows us to encrypt a vector of messages to all n parties. Of course, we can always implement an MRE scheme by using a standard encryption scheme applied separately to each of the n plaintexts. The goal in designing an MRE is to get ciphertexts that are more compact and more efficiently computable.

The key set-up is the same as in Section 2. That is, we assume that all parties have public keys $pk_1, \dots, pk_n$. Each honest party P_j will generate a public/secret-key pair independently using a key-generation algorithm. Corrupt parties can choose their keys arbitrarily — even depending on the public keys of the honest parties. Note that unlike in other works on MRE schemes, such as [BBS03], we do *not* require users to prove possession of their secret keys.

The MRE scheme we use is a simplified version of the MEGa scheme in [GS22], in that our MRE scheme does not support decryption proofs. For simplicity, we also use the same group $\mathbb{G}$ as above for decryption, although this is not necessary. We assume the message space $\mathcal{M}$ is the set of all bit strings of some fixed length. We use a random oracle H_{enc} whose output space is $\mathcal{M}$.

Each user P_j has a public key $u_j := g^{\alpha_j}$ and random secret key $\alpha_j \in \mathbb{Z}_q$. To encrypt a vector $(m_1, \dots, m_n) \in \mathcal{M}^n$ with associated data ad , we choose $\beta \in \mathbb{Z}_q$ at random, compute $v := g^\beta$, and then

$$w_j := u_j^\beta, \quad e_j := m_j \oplus H_{\text{enc}}(ad, j, u_j, v, w_j) \in \mathcal{M} \quad (j = 1, \dots, n).$$

The ciphertext is

$$(v, e_1, \dots, e_n) \in \mathbb{G} \times \mathcal{M}^n.$$

Decryption works in the obvious way.

This MRE scheme is AD-only CCA secure in the random oracle model under the I^2CDH assumption, which is defined in [GS22]. This is a falsifiable variant of the gap-DH assumption, where the adversary is given $u = g^\alpha$ and $v = g^\beta$, and is tasked to compute $w = g^{\alpha\beta}$ given access to a DDH oracle that decides if a given triple is a Diffie-Hellman triple (that is, of the form $(g^\kappa, g^\lambda, g^{\kappa\lambda})$). However, in the I^2CDH attack game, each such triple must be of the form $(u, \cdot, \cdot)$ or $(v, \cdot, \cdot)$. The analysis in [GS22] does not quite exactly cover the situation here, but it is straightforward to adapt it to do so. In the security proof, access to the DDH oracle with u is used to simulate the decryption oracle (with adversarial ciphertexts and honest public keys), while access to the DDH oracle with v is used to simulate the encryption oracle (with adversarial public keys).

The I^2CDH assumption is just a slight generalization of the ICDH assumption, which is used to analyze standard hashed ElGamal encryption (aka, DHIES or ECIES). In the ICDH attack game, each input to the DDH oracle must be of the form $(u, \cdot, \cdot)$. We note that we could get by using just the ICDH assumption in our MRE scheme if we insisted that users to prove possession of their secret keys (as in [BBS03]). However, there seems to be no reason to believe that the ICDH assumption holds but the I^2CDH assumption does not.

The nice thing about this MRE scheme is that ciphertexts are essentially half the size (and take half as much time to compute) than what we would get by using a standard ElGamal-type scheme applied separately to each plaintext.

4 A simple VESS scheme

Our VESS scheme is an optimized and generalized version of a verifiable encryption scheme in [ASW00].¹ It uses the MRE scheme in Section 3. The public/secret keys for the VESS are the same as for the MRE scheme. The message space $\mathcal{M}$ for the MRE scheme is the set of all bit strings of length equal to the length of $q - 1$, which encode elements of $\mathbb{Z}_q$.

The VESS scheme is parameterized by a repetition parameter N and a collection $\mathcal{S}$ of nonempty subsets of $[N]$. For example, we may choose $\mathcal{S}$ to be all nonempty subsets or all subsets of some given size. As we will see, there are time/space/security trade-offs in the choice of $\mathcal{S}$.

We will assume a random oracle H_{choose} whose output is a subset-seed s that can be used to derive a subset $S \in \mathcal{S}$. The derivation algorithm *derive* that maps s to $S \in \mathcal{S}$ should be (computationally) pseudorandom — that is, it should be hard to distinguish random S from *derive*(s), where s is a random subset-seed.

We will assume a random oracle H_{expand} that takes as input an expansion-seed r and expands it to $(\beta, \omega') \in \mathbb{Z}_q \times \mathbb{Z}_q[x]_{<t}$. The expansion-seed space should be large enough to ensure that (β, ω') is indistinguishable from random.

¹That scheme verifiably encrypts homomorphic preimages, such as discrete logarithms. One could use that scheme directly as a black box. However, we get a significantly more efficient scheme by adapting the basic technique to verifiable secret sharing.

We will assume a random oracle H_{compress} whose output space is large enough to prevent collisions.

We first describe a Sigma protocol where the verifier's challenge is a random subset-seed s . The actual encryption and verification algorithms are obtained by applying the Fiat-Shamir heuristic, using H_{choose} to generate the challenge.

The input to the prover is $\omega \in \mathbb{Z}_q[x]_{<t}$ and associated data ad . The input to the verifier is ad .

1. For $k \in [N]$, prover chooses a random expansion-seed r_k , and computes

- (a) $(\beta_k, \omega'_k) \leftarrow H_{\text{expand}}(ad, k, r_k)$,

- (b) $(v_k, e_{k1}, \dots, e_{kn})$, where

$$v_k := g^{\beta_k}, \quad w_{kj} := u_j^{\beta_k}, \quad e_{kj} := \langle \omega'(j) \rangle \oplus H_{\text{enc}}(ad, k, j, u_j, v_k, w_{kj}),$$

$$\text{for } j \in [n],$$

- (c) $\mathbf{C}'_k \leftarrow g^{\omega'_k}$,

- (d) $\theta_k \leftarrow (\mathbf{C}'_k; v_k, e_{k1}, \dots, e_{kn})$.

2. Prover sends $\mathbf{C}$ and $h := H_{\text{compress}}(ad, \theta_1, \dots, \theta_N)$ to verifier, where

$$\mathbf{C} := g^\omega$$

and

$$\theta_k := (\mathbf{C}'_k; v_k, e_{k1}, \dots, e_{kn}),$$

for $k \in [N]$.

3. Verifier generates a random subset-seed s and sends this to the prover.
4. Prover derives a subset $S \in \mathcal{S}$ from s , and computes ρ_k for $k \in [N]$ as follows.

- (a) If $k \in S$, then

$$\rho_k := (\omega''_k; v_k, e_{k1}, \dots, e_{kn}),$$

$$\text{where } \omega''_k := \omega'_k + \omega.$$

- (b) If $k \notin S$, then

$$\rho_k = r_k.$$

Prover sends $(\rho_1, \dots, \rho_N)$ to the verifier.

5. Verifier accepts iff $H_{\text{compress}}(ad, \theta_1, \dots, \theta_N) = h$, where θ_k for $k \in [N]$ is computed as follows.

- (a) If $k \in S$, then

$$\theta_k := (g^{\omega''_k} / \mathbf{C}; v_k, e_{k1}, \dots, e_{kn}),$$

$$\text{where } \rho_k = (\omega''_k; v_k, e_{k1}, \dots, e_{kn}).$$

- (b) If $k \notin S$, then θ_k is computed from r_k using the same logic of the prover in Step 1, where $\rho_k = r_k$.

When we turn the above into a non-interactive proof using Fiat-Shamir, the dealing output by the encryption algorithm consists of $(\mathbf{C}, h, \rho_1, \dots, \rho_N)$, where the subset-seed s is computed as

$$s \leftarrow H_{\text{choose}}(ad, \mathbf{C}, h).$$

The encryption algorithm generates this dealing by running Steps 1–4 of the interactive protocol (but deriving s from a hash in Step 3). The verification algorithm just runs Step 5 of the interactive protocol.

The decryption algorithm takes as input a dealing $(\mathbf{C}, h, \rho_1, \dots, \rho_N)$, together with associated data ad , recipient index $j \in [n]$, and that recipient’s secret key sk_j . It then does the following, for each $k \in S$:

1. decrypt e_{kj} using sk_j , obtaining a scalar $\mu'_{kj} \in \mathbb{Z}_q$;
2. compute $\mu_{kj} := \omega''_k(j) - \mu'_{kj}$;
3. test if $g^{\mu_{kj}} = \mathbf{C}^{(j)}$;
4. if this test succeeds, output μ_{kj} and halt.

If this loop does not succeed in producing an output for any $k \in S$, the decryption algorithm outputs **fail**.

NOTES:

1. We input to each hash function the associated data ad . Also for H_{expand} and H_{enc} , we input the associated iteration index k . For H_{enc} , this corresponds to using associated data (ad, k) in the MRE scheme. Theoretically, we could have omitted the iteration index k from all hash inputs, and we could have omitted ad from all inputs *except* H_{enc} — the ad input to H_{enc} plays an essential role in the MRE security analysis. However, including these extra inputs is just good security practice, as it provides better “domain separation”, which can lead to simpler and more efficient security reductions.

4.1 Security properties

4.1.1 Soundness

To analyze soundness, let us first consider the probability of breaking soundness in the interactive setting, where the subset-seed s is generated at random. Here, we are modeling H_{expand} and H_{compress} as random oracles. We do not need to model H_{enc} as a random oracle here. We also assume the adversary is given the secret keys of all honest parties. So the adversary’s interactive soundness advantage is the probability that the adversary can produce a dealing that makes the verifier accept, but which fails to decrypt for some honest recipient P_j . Assuming q and the output space of H_{compress} are large, and the algorithm that derives S from s is pseudorandom, this probability is negligibly close to $1/S$. Indeed, to win, the prover must essentially guess $S \in \mathcal{S}$ by the time it finishes Step 2 of the protocol.

In the non-interactive setting, the protocol (like any protocol derived via Fiat-Shamir) is subject to a “grinding attack”, and the adversary’s advantage in breaking soundness is

at most (negligibly close to) $Q_{\text{choose}}/\mathcal{S}$, where Q_{choose} is the number of queries made to the random oracle H_{choose} made by the adversary.

NOTES:

1. The effectiveness of such a grinding attack can be reduced by the standard technique of adding a standard “proof of work” (aka “grinding”) mechanism to the protocol.

4.1.2 Privacy

We can adopt the same simulation-based definition of AD-only CCA security presented in [ASW00] to this setting. Here, the adversary specifies the set of corrupt parties, and keys for all parties are generated. Then, the adversary makes a series of queries to an encryption and decryption oracle. In an encryption query, the adversary submits (ω, ad) to an encryption oracle that produces a dealing. In a decryption query, we give the adversary direct access to a decryption oracle for the underlying MRE scheme. Queries to this oracle specify an MRE ciphertext, a recipient index j , and associated data of the form (ad, k) , where $k \in [N]$. The MRE ciphertext is decrypted under secret key sk_j and associated data (ad, k) . The only restriction is that if the encryption oracle is queried as $(\cdot, ad)$, then subsequent decryption queries of the form $(ad, \cdot)$ are not allowed. The adversary also has access to an oracle representing the random oracle H_{compress} . We do not need to model H_{expand} , H_{choose} , or H_{enc} as random oracles here. However, we do need to assume that H_{expand} is a secure pseudorandom generator.

The above describes the “real world”. In the “ideal world”, there is a simulator that simulates the decryption and random oracle queries. However, in each encryption query (ω, ad) , the simulator is given access only to ad , $\mathbf{C} = g^\omega$, and $\{\omega(j)\}_{j \in \mathcal{C}}$.

Privacy says that there is an efficient simulator such that no efficient adversary can effectively distinguish between the real and ideal worlds.

The design of the simulator of our VESS scheme is a standard application of random-oracle programming. For a given encryption query (ω, ad) , the simulator just chooses a random output h for H_{compress} , which (together with ad and $\mathbf{C}$) determines S . Then, using S , it can compute appropriate values for θ_k and ρ_k for all $k \in N$, and then program H_{compress} so that $H_{\text{compress}}(ad, \theta_1, \dots, \theta_N) = h$.

- For $k \notin S$, it chooses r_k as a random expansion-seed, and then computes θ_k and ρ_k following the prover’s algorithm.
- For $k \in S$, it chooses a random polynomial $\omega''_k \in \mathbb{Z}_q[x]_{<t}$. Then it computes an MRE encryption $(v_k, e_{k1}, \dots, e_{kn})$, where
 - for each honest recipient, it encrypts a dummy value instead of $\omega'(j)$ (this is safe, by the AD-only CCA security of the MRE scheme), and
 - for each corrupt recipient P_j , it encrypts $\omega'_k(j)$ computed as $\omega''_k(j) - \omega(j)$ (this can be done, as the simulator was given $\omega(j)$ and generated ω''_k itself).

The simulator also computes $\mathbf{C}'_k := g^{\omega''_k}/\mathbf{C}$, where $\mathbf{C} = g^\omega$ was given to the simulator. Finally the simulator sets $\theta_k = (\mathbf{C}'_k; v_k, e_{k1}, \dots, e_{kn})$ and $\rho_k = (\omega''_k; v_k, e_{k1}, \dots, e_{kn})$.

This ideal world simulation deviates from the real world only insofar as (a) H_{expand} fails to be a pseudorandom generator, (b) the MRE scheme fails to provide AD-only CCA security, and (c) programming the random oracle H_{compress} fails (which happens with negligible probability).

NOTES:

1. We naturally could have defined privacy so that the adversary is given access to a decryption oracle for VESS, rather than the underlying MRE. By giving the adversary direct to the decryption oracle for the MRE, we can be sure that the scheme is not subject to a crude timing attack in which the VESS decryption algorithm reveals which MRE decryption attempts failed, or even the results of those attempts.
2. We have defined soundness and simulation-based privacy as two distinct notions, rather than any form of “simulation soundness”, which intertwines the two. However, this is sufficient for typical applications — see Appendix A for details on how these security properties can be used to construct a secure DKG protocol.

4.1.3 Correctness

Correctness is clear.

4.2 Cost estimates

A dealing consists of

$$|S| \cdot (n + t + 1) + (N - |S|) + t + 1$$

objects, each of which is a group element, scalar, bit string of scalar length, expansion-seed, or hash function output. For purposes of estimating sizes, we can take each of these objects to be 32 bytes long.

For computation costs, we count only exponentiations.

- T_{gconst} — time to exponentiation to the base g using a constant-time algorithm;
- T_{gvar} — time to exponentiation to the base g using a variable-time algorithm;
- T_{aconst} — time to exponentiation to any base using a constant-time algorithm;
- T_{avar} — time to exponentiation to any base using a variable-time algorithm.

We give here benchmarks using `libsecp256k1`² on an Apple M1 Max. We state benchmarks for these operations that can be implemented directly using functions in the documented API. However, we shall also indicate how these functions could be fairly easily modified to better suit our needs and achieve better performance.

- T_{gconst} : 16 μ s using `ecmult_gen`.

²See <https://github.com/bitcoin-core/secp256k1>.

- T_{gvar} : $16\mu\text{s}$ using `ecmult_gen`.

One can use the same logic as `ecmult_gen` but without the overhead constant-time table lookups and constant-time additions. This would cost 64 `group_add_affine_var` operations, each of which takes $0.172\mu\text{s}$, for a total of $11\mu\text{s}$. We will use this value in the estimates below.

- T_{aconst} : $31\mu\text{s}$ using `ecmult_const`.

By precomputing appropriate tables (based on the public keys of the recipients), we can use the same logic as `ecmult_gen`.³ This would then take $16\mu\text{s}$. We will use this value in the estimates below.

T_{avar} : $24\mu\text{s}$ using `ecmult_1p`.

Using the same precomputed tables as for T_{aconst} , we can use the same logic as `ecmult_gen` but without the overhead constant-time table lookups and constant-time additions. This would cost 64 `group_add_affine_var` operations, each of which takes $0.172\mu\text{s}$, for a total of $11\mu\text{s}$.

The above timing estimates do not take into account the time to convert from Jacobian to affine coordinates. However, all of these conversions can be done in large batches, and using standard batching techniques, the cost of these conversions can safely be ignored.

Encryption cost is

$$N \cdot ((t+1) \cdot T_{\text{gconst}} + n \cdot T_{\text{aconst}}) + t \cdot T_{\text{gconst}}.$$

Verifier cost is

$$|S| \cdot T_{\text{avar}} + (N - |S|) \cdot ((t+1) \cdot T_{\text{gvar}} + n \cdot T_{\text{avar}}).$$

Of course, all computations are trivially parallelizable, and so if multiple cores are available, these can be readily exploited to significantly reduce computation times. In a typical DKG application, the most important computation cost is verifier cost, as each party will have to play the role of verifier many times, but the role of dealer only once. We do not estimate decryption times — in typical DKG applications, these will not contribute significantly to the overall running time.

As suggested in [TZ21], we can choose $\mathcal{S}$ to be the set of all subsets of some fixed size M , and then choose various (N, M) pairs to get 128-bit security. For concreteness, let us also assume that $f = 4$, $n = 3f + 1 = 13$, and $t = 5$.

1. $(N, M) = (132, 64)$.

- (a) A dealing has byte length

$$32 \cdot (64 \cdot (13 + 6) + 68 + 6) = 41280,$$

so 41KB.

³Note that the implementation of `ecmult_gen` requires a point U whose discrete logarithm with respect to the base point is unknown. Since the base points here are public keys, some of which may be adversarially chosen, we can compute U as a hash of these public keys.

(b) Encryption time in μs is

$$132 \cdot (6 \cdot 16 + 13 \cdot 16) + 5 \cdot 16 = 40208,$$

so 40ms.

(c) Verifier time in μs is

$$64 \cdot 11 + 68 \cdot (6 \cdot 11 + 13 \cdot 11) = 14916,$$

so 15ms.

2. $(N, M) = (250, 30)$.

(a) A dealing has byte length

$$32 \cdot (30 \cdot (13 + 6) + 220 + 6) = 25472,$$

so 25KB.

(b) Encryption time in μs is

$$250 \cdot (6 \cdot 16 + 13 \cdot 16) + 5 \cdot 16 = 76080,$$

so 76ms.

(c) Verifier time in μs is

$$30 \cdot 11 + 220 \cdot (6 \cdot 11 + 13 \cdot 11) = 46310,$$

so 46ms.

Note that if one chooses not to use the modifications to `libsecp256k1` suggested above, and just use the documented API, the above encryption and verification times would roughly double.

5 Related Work

As we have already mentioned, our VESS scheme is a generalization and optimization of the cut-and-choose verifiable encryption scheme in [ASW00]. That scheme verifiably encrypts a single discrete logarithm to a single user. It could be used directly as a black box to implement a VESS, but the resulting VESS scheme would be less efficient and more complicated than the VESS scheme presented here. Subsequent to [ASW00], another cut-and-choose scheme was presented in [CD00]; however, that scheme is a bit less efficient than the one in [ASW00]; in addition, it does not provide CCA security, which is needed in typical applications.

Besides verifiably encrypting a discrete logarithm via a cut-and-choose construction, there is also the (folklore?) construction of encrypting a discrete logarithm bit-by-bit (or a few bits at a time) — see, for example, Exercise 20.11 of [BS23]. This type of construction will generally be much less efficient and really no simpler than cut-and-choose.

Other relevant work on verifiable encryption of discrete logarithms includes [CS03] and [TZ21].⁴ For [CS03], we estimate that using a 3072-bit RSA modulus (and employing the usual trick for compressing the Fiat-Shamir transform of a Sigma protocol to a hash output and response) each ciphertext+proof is about 4KB (4 Paillier group elements, 3 scalars, and one hash output). So applying this directly to get a VESS, with $n = 13$, each dealing would be about 52KB. In [TZ21], the smallest ciphertext+proof reported is also about 4KB. So, again, applying this directly to get a VESS, with $n = 13$, each dealing would also be about 52KB. In both cases, this is more than the size of 25KB for a dealing that we reported above in Section 4.2. That said, there may be ways to optimize the schemes in [CS03] and [TZ21] for the application to VESS (for example, by employing a more efficient MRE scheme, rather than using independent encryptions).

For recent work on VESS, see [Gro21] and [KMM⁺23]. Both of these papers give VESS schemes that are more efficient than our simple-minded VESS scheme. For example, the scheme in [KMM⁺23] has dealings of size 3KB for $n = 13$, compared to the size of 25KB for a dealing that we reported above in Section 4.2. Also, extrapolating from tables in [KMM⁺23], we estimate the time to verify a dealing for that scheme to be 17ms (on an Intel Xeon 2.8GHz). Compare this to 46ms for instance of our scheme with 25KB-sized dealings, or 15ms for the instance with 41KB-sized dealings. This is not quite an apples-to-apples comparison, as we estimated the running time of our VESS scheme on an Apple M1 Max, but it should not be too far off from one. (We did not succeed in building code associated with [KMM⁺23] on our machine.)

We note that [Gro21] and [KMM⁺23] rely on much more sophisticated cryptographic algorithms and less well established cryptographic assumptions (and proofs of possession of decryption keys). For example, [Gro21] relies on a bespoke decisional assumption for a pairing-friendly elliptic curve (justified by the generic group model), and [KMM⁺23] relies on fairly recently proposed hardness assumptions on ideal class groups of imaginary quadratic fields. The paper [KMM⁺23] also contains a good overview of the literature on VESS.

Finally, we note that the VESS scheme in [KMM⁺23] is not shown to provide any kind of CCA security, which is typically required for applications such as DKG. That said, [KMM⁺23] does claim a DKG protocol built using this VESS following the usual paradigm of having each party generate a dealing and then combining an agreed-upon set of such dealings. However, other protocols in the literature that follow this same paradigm, including [Gro21], require a CCA-secure VESS scheme in their proofs of security. See Appendix A for a detailed review of these proof techniques. For this reason, we have some reservations about the security of the DKG scheme in [KMM⁺23].

⁴Contrary to what is reported in [TZ21], the scheme in [CS03] works over an arbitrary group and does not require range proofs.

References

- [ASW00] N. Asokan, V. Shoup, and M. Waidner. Optimistic fair exchange of digital signatures. *IEEE J. Sel. Areas Commun.*, 18(4):593–610, 2000. Also in *Eurocrypt 1998* (<https://link.springer.com/chapter/10.1007/BFb0054156>) and at <https://shoup.net/papers/fex.pdf>.
- [BBS03] M. Bellare, A. Boldyreva, and J. Staddon. Randomness re-use in multi-recipient encryption schemes. In Y. Desmedt, editor, *Public Key Cryptography - PKC 2003, 6th International Workshop on Theory and Practice in Public Key Cryptography, Miami, FL, USA, January 6-8, 2003, Proceedings*, volume 2567 of *Lecture Notes in Computer Science*, pages 85–99. Springer, 2003.
- [Bol03] A. Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In Y. Desmedt, editor, *Public Key Cryptography - PKC 2003, 6th International Workshop on Theory and Practice in Public Key Cryptography, Miami, FL, USA, January 6-8, 2003, Proceedings*, volume 2567 of *Lecture Notes in Computer Science*, pages 31–46. Springer, 2003.
- [Bra87] G. Bracha. Asynchronous byzantine agreement protocols. *Inf. Comput.*, 75(2):130–143, 1987.
- [BS23] D. Boneh and V. Shoup. *A Graduate Course in Applied Cryptography (v0.6)*. 2023. Available at <https://toc.cryptobook.us/>.
- [Can20] R. Canetti. Universally composable security. *J. ACM*, 67(5):28:1–28:94, 2020. Also at <https://eprint.iacr.org/2000/067>.
- [CD00] J. Camenisch and I. Damgård. Verifiable encryption, group encryption, and their applications to separable group signatures and signature sharing schemes. In T. Okamoto, editor, *Advances in Cryptology - ASIACRYPT 2000, 6th International Conference on the Theory and Application of Cryptology and Information Security, Kyoto, Japan, December 3-7, 2000, Proceedings*, volume 1976 of *Lecture Notes in Computer Science*, pages 331–345. Springer, 2000. Also at <https://eprint.iacr.org/1999/008> and <https://www.brics.dk/RS/98/32/>.
- [CR03] R. Canetti and T. Rabin. Universal composition with joint state. In D. Boneh, editor, *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*, volume 2729 of *Lecture Notes in Computer Science*, pages 265–281. Springer, 2003. Also at <https://eprint.iacr.org/2002/047>.
- [CS03] J. Camenisch and V. Shoup. Practical verifiable encryption and decryption of discrete logarithms. In D. Boneh, editor, *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*, volume 2729 of

- Lecture Notes in Computer Science*, pages 126–144. Springer, 2003. Also at <https://eprint.iacr.org/2002/161>.
- [Gro21] J. Groth. Non-interactive distributed key generation and key resharing. Cryptology ePrint Archive, Report 2021/339, 2021. <https://eprint.iacr.org/2021/339>.
 - [GS22] J. Groth and V. Shoup. Design and analysis of a distributed ECDSA signing service. Cryptology ePrint Archive, Report 2022/506, 2022. <https://eprint.iacr.org/2022/506>.
 - [KMM⁺23] A. Kate, E. V. Mangipudi, P. Mukherjee, H. Saleem, and S. A. K. Thyagarajan. Non-interactive VSS using class groups and application to DKG. Cryptology ePrint Archive, Paper 2023/451, 2023. <https://eprint.iacr.org/2023/451>.
 - [LS25] T. Locher and V. Shoup. Improving the round complexity of MiniCast. Cryptology ePrint Archive, Paper 2025/779, 2025. <https://eprint.iacr.org/2025/779>.
 - [SG02] V. Shoup and R. Gennaro. Securing threshold cryptosystems against chosen ciphertext attack. *J. Cryptol.*, 15(2):75–96, 2002. Also at *Eurocrypt 1998*.
 - [Sho24] V. Shoup. A theoretical take on a practical consensus protocol. Cryptology ePrint Archive, Paper 2024/696, 2024. <https://eprint.iacr.org/2024/696>.
 - [Sho25] V. Shoup. The many faces of Schnorr: a toolkit for the modular design of threshold Schnorr signatures. *IACR Communications in Cryptology*, 2(1), 2025. Also at <https://eprint.iacr.org/2023/1019>.
 - [SS24] V. Shoup and N. P. Smart. Lightweight asynchronous verifiable secret sharing with optimal resilience. *J. Cryptol.*, 37(3):27, 2024. Also at <https://eprint.iacr.org/2023/536>.
 - [TZ21] A. Takahashi and G. Zaverucha. Verifiable encryption from MPC-in-the-head. Cryptology ePrint Archive, Paper 2021/1704, 2021. <https://eprint.iacr.org/2021/1704>, also in *IACR Communications in Cryptology* (<https://cic.iacr.org/p/1/1/3>).

A Realizing ideal functionalities using VESS

One of the main motivations for a VESS scheme is that it can be used to easily implement a **verifiable secret sharing (VSS)** protocol. There are many different flavors of VSS. We specify the particular flavor we are interested in by giving a corresponding ideal functionality, as in the Universal Composability framework [Can20]. We then show how to securely realize this functionality using a VESS scheme. Finally, we discuss how to use a VSS protocol to build a DKG protocol.

A.1 System setup

As usual, we have n parties $P_1, \dots, P_n$, at most f of which are corrupted. We assume the adversary statically corrupts a set $\mathcal{C} \subseteq [n]$ at the very beginning of the attack, and that $\mathcal{H} := [n] \setminus \mathcal{C}$. We assume asynchronous, authenticated point-to-point communication channels. Because of asynchrony, we shall assume $f < n/3$, as this is required for many protocols in this setting. We also have a reconstruction parameter t , and we assume $t \leq n - f$.

A.2 Ideal functionality for VSS

We now describe the ideal functionality $\mathcal{F}_{\text{VSS}}$ for VSS. A single instance of $\mathcal{F}_{\text{VSS}}$ is parameterized by an index i , which specifies that party P_i is the dealer for this instance — by convention, we assume that the value i is encoded in the session ID of that instance. The logic of $\mathcal{F}_{\text{VSS}}$ is as follows:

Party P_i inputs $\omega \in \mathbb{Z}_q[x]_{<t}$ to $\mathcal{F}_{\text{VSS}}$, who computes $\mathbf{C} \leftarrow g^\omega$, sends

$$(\mathbf{C}, \{\omega(j)\}_{j \in \mathcal{C}})$$

to the adversary, and sends $(\mathbf{C}, \omega(j))$ to each honest party P_j as a delayed output.

By *delayed output*, we mean the adversary informs $\mathcal{F}_{\text{VSS}}$ when to send an output to some particular P_j . This delayed output mechanism allows us to model asynchronous communication.

We require that a VSS protocol is *complete*. We do not attempt to model this property as an ideal functionality. Rather, as in [SS24], for example, we define completeness as a property of the protocol. In a nutshell, completeness says that if the dealer is honest, or if some honest party generates an output, then eventually all honest parties generate an output. Here, *eventually* means if and when all protocol messages between honest parties have been delivered (which is completely under control of the adversary).

Notice that this definition of VSS says that:

- for an honest dealer: some information about their input ω leaks to the adversary, namely, $\mathbf{C} := g^\omega$;
- for a corrupt dealer: they must fully commit to their input ω before the first honest party generates an output, in the sense that ω can effectively be *extracted*.

A.3 Implementing VSS using VESS

It is trivial to implement VSS using a VESS scheme: the dealer generates a dealing $(\mathbf{C}, \chi)$ for ω and *reliably broadcasts* this dealing to all parties. Upon receiving this dealing, each party checks if the dealing is valid, and if so, decrypts and outputs its share. For reliable broadcast, one can use Bracha’s protocol [Bra87], or a more modern, communication-efficient variant (such as [LS25]).

Let us call this simple protocol Π_{VSS} . To prove that Π_{VSS} securely realizes $\mathcal{F}_{\text{VSS}}$, we must first observe that Π_{VSS} actually makes use of a PKI, which is shared across many instances of the protocol. So we must consider the multi-instance version of Π_{VSS} and directly prove that it securely realizes the multi-instance version of $\mathcal{F}_{\text{VSS}}$ [CR03], rather than rely on the usual UC composition theorem [Can20].

To this end, we need to be a bit more explicit about how the PKI is set up. As already mentioned in Section 2, each honest party P_j will generate a public/secret-key pair independently using a key-generation algorithm. Corrupt parties can choose their keys arbitrarily — even depending on the public keys of the honest parties. This can be modeled by an ideal functionality $\mathcal{F}_{\text{PKI}}$:

- $\mathcal{F}_{\text{PKI}}$ generates public/secret-key pairs for all honest parties, and gives these to the adversary.
- The adversary then chooses public keys for all of the corrupt parties, and gives these to $\mathcal{F}_{\text{PKI}}$.
- $\mathcal{F}_{\text{PKI}}$ then gives to each honest party its own secret key, as well as the list of all public keys, as a delayed output.

In addition to $\mathcal{F}_{\text{PKI}}$, we also need an ideal functionality representing any random oracles needed by the VESS scheme. (For the VESS scheme presented in Section 4, the only hash function that needs to be modeled explicitly as a random oracle functionality is H_{compress} .) Thus, the multi-instance version of Π_{VSS} is a *hybrid* protocol that uses the ideal functionality representing the random oracles, as well as $\mathcal{F}_{\text{PKI}}$.

In the multi-instance version of Π_{VSS} , each instance uses the same PKI, but is identified by a unique **subsession ID (SSID)**. A crucial detail we have left out of the above description of Π_{VSS} is that *this subsession ID is used as the associated data of the corresponding VESS*. This is consistent with the recommended setting for the associated data as discussed in Section 2.

We sketch the main idea of the proof that the multi-instance version of Π_{VSS} securely realizes the multi-instance version of $\mathcal{F}_{\text{VSS}}$.

1. We start with an execution representing the real world, in which a “dummy adversary” follows the instructions provided by the environment, and sends back to the environment all protocol messages generated by honest parties (or outputs generated by $\mathcal{F}_{\text{PKI}}$ and the random oracle functionality).
2. We then modify the execution so that when the dealer of any VSS instance is corrupt, we *extract* its input $\omega \in \mathbb{Z}_q[x]_{<t}$ at the time the first honest party has received and

validating the dealing. This is done by decrypting the dealing on behalf of *all* honest parties and then interpolating.

This is where we use the soundness property of the VESS (which ensures the decryption succeeds) and the assumption that $n - f \geq t$ (which ensures that we have enough points to interpolate).

3. We then modify the execution so that when the dealer of any VSS instance is honest, we do not bother decrypting the dealing on behalf of honest parties, but just output that party's share directly to the environment.

For this, we rely on the correctness property of the VESS scheme. So now, the only decryptions are those done in Step 2.

4. We then modify the execution so that when the dealer of any VSS instance is honest, we use the simulator guaranteed by the privacy property of the VESS scheme to generate a simulated dealing. (For the VESS scheme presented in Section 4, that simulator needs to program the random oracle for H_{compress} .)

Crucially, the conventions regarding associated data guarantee that the associated data values of the dealings decrypted in Step 2 are disjoint from the associated data values of the simulated dealings in this step, and hence AD-only CCA security property guarantees that the environment will not be able to notice this change.

After the modification made in Steps 1–4, the simulator for the ideal world is trivial to construct. Inputs extracted in Step 2 are given to the ideal functionality on behalf of corrupt parties, and the simulated dealings generated in Step 4 are used to simulate all data sent back to the environment on behalf of the “dummy adversary”.

A.4 From VSS to DKG

It is easy to build a DKG protocol from a VSS protocol. We simply have each party run an instance of the VSS protocol as a dealer with random input polynomial. Of course, while honest parties will choose random input polynomials, corrupt parties may choose their inputs arbitrarily. We then agree on a set $\mathcal{I}$ of $f + 1$ dealers whose VSS instances have successfully completed. Suppose those VSS instances yield $\mathbf{C}_i \in \mathbb{G}^t$ and $\omega_i \in \mathbb{Z}_q[x]_{<t}$ for $i \in \mathcal{I}$, so each party P_j has received VSS outputs $(\mathbf{C}_i, \omega_i(j))$ for $i \in \mathcal{I}$. Then the output generated by each party P_j is $\prod_i \mathbf{C}_i$ and $\sum_i \omega_i(j)$.

To agree upon the set $\mathcal{I}$, we need to run a protocol for **agreement on a common set (ACS)**. As we have structured the problem, the type of ACS protocol we need is an *index-oriented* ACS protocol, which outputs a set of parties (in this case, dealers) rather than a *value-oriented* ACS protocol, which outputs a set of values (one per party). For a formal definition of an index-oriented ACS protocol, see, for example, [Sho24]. Note that if we implement VSS using a VESS scheme, then instead of reliably broadcasting a dealing and then running an index-oriented ACS protocol, we can also disseminate these dealings directly using a value-oriented ACS protocol.

Because the VSS protocol leaks the values $\mathbf{C}_i$ of honest parties P_i , this DKG protocol allows the adversary to bias the result to a certain degree. One can characterize this bias by showing that this protocol securely realizes the following ideal functionality, $\mathcal{F}_{\text{BiasDKG}}$:

- $\mathcal{F}_{\text{BiasDKG}}$ chooses $\omega \in \mathbb{Z}_q[x]_{<t}$ at random, computes $\mathbf{C} := g^\omega \in \mathbb{G}^t$, and gives

$$(\mathbf{C}, \{\omega(j)\}_{j \in \mathcal{C}})$$

to the adversary.

- Later, the adversary chooses $\epsilon \in \mathbb{Z}_q^*$ and $\omega' \in \mathbb{Z}_q[x]_{<t}$ and sends these to $\mathcal{F}_{\text{BiasDKG}}$.
- $\mathcal{F}_{\text{BiasDKG}}$ computes $\omega'' \leftarrow \omega + \omega' \in \mathbb{Z}_q[x]_{<t}$ and $\mathbf{C}'' \leftarrow g^{\omega''} \in \mathbb{G}^t$, and sends $(\mathbf{C}'', \omega''(j))$ to each honest party P_j as a delayed output.

This can be proved following the same random-self-reducibility argument as in, for example, Appendix C.5 of [Sho25]. For some applications, such as threshold BLS signatures [Bol03] or threshold ElGamal decryption [SG02], such bias is acceptable. However, for other applications, such as threshold ECDSA, such bias may completely break the system (see Section 3.6 of [GS22]). In such cases, an alternative approach based on a VESS scheme is to use a “private bulletin board”, as discussed in Section 22.4 of [BS23].